

Stochastic Differential Equations

A *differential equation* describes how something changes from moment to moment: a rule like "the population grows by 3% per year". A *stochastic* differential equation (SDE) adds a random push to that rule at every instant. SDEs model stock prices, interest rates, molecules and neurons, and they are what Itô calculus was built to handle.

5.1 Euler–Maruyama

Most SDEs can't be solved with a formula, so computers solve them step by step: from where you are now, add the predictable part of the change, add a random kick, repeat. This recipe is the *Euler–Maruyama* method. To check it, we use an SDE that *can* be solved exactly (a model of a stock price) and compare the two, driven by the same Brownian path.

Predict first: To cut the average error in half, how many times more steps will you need?

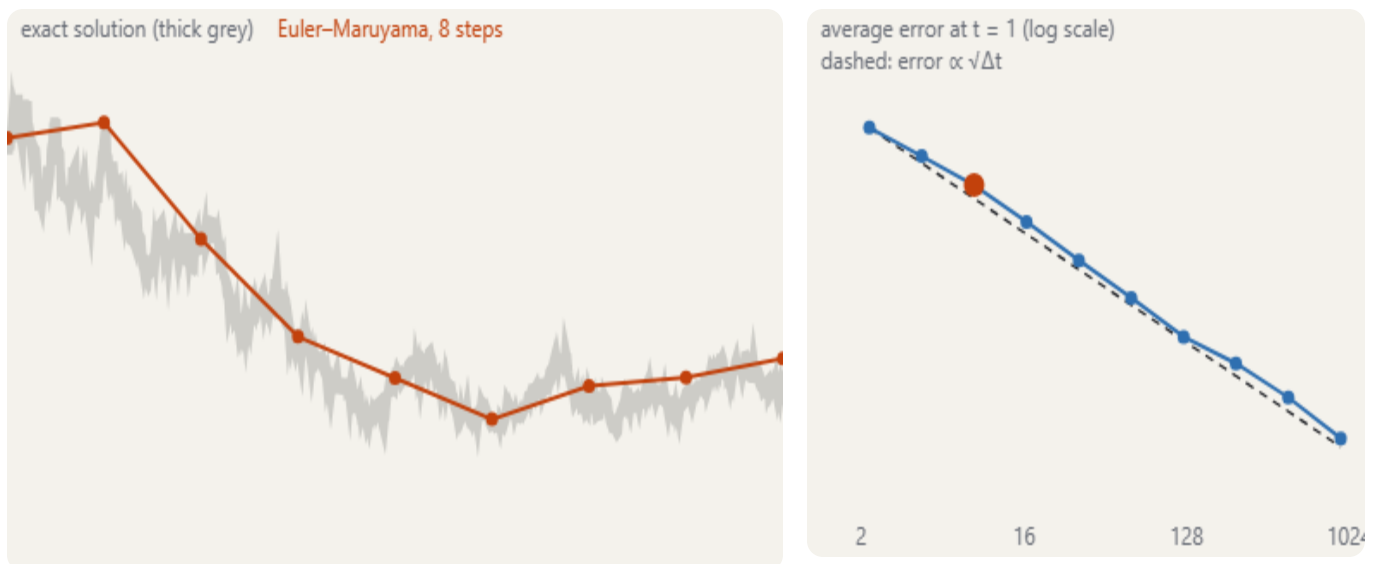
Answer: "4×". The error shrinks like $\sqrt{\Delta t}$, so halving it needs Δt four times smaller: 4× the steps.

Drag the number of steps. Left: the exact solution (thick grey) and the step-by-step one (orange). Right: the average error over 500 paths for every step count.

$$X_{k+1} = X_k + a(X_k) \Delta t + b(X_k) \Delta W_k \quad \text{here: } a(x) = \mu x, \quad b(x) = \sigma x$$

- X_k The computed value after k steps (the orange dots). It starts at $X_0 = 1$.
- Δt The size of one time step: $1/N$ for N steps.
- $a(x)$ The *drift*: the predictable part of the change per unit of time. Here $a(x) = \mu x$ with $\mu = 0.2$: growth of 20% per unit of time, like interest.
- $b(x)$ The *volatility*: how strongly the random kicks act. Here $b(x) = \sigma x$ with $\sigma = 0.8$: kicks proportional to the current value, so the price moves in percentage terms.
- ΔW_k The Brownian path's rise during step k : a bell-curve random number with variance Δt (section 1.2).
- exact solution For this model Itô's lemma gives a formula: $X_t = e^{(\mu - \sigma^2/2)t + \sigma W_t}$. The $-\sigma^2/2$ is the Itô correction from Chapter 4.

In plain words: each step freezes the drift and the volatility at their values at the start of the step. That is Itô's left-end rule again (section 3.1), which is why this method matches Itô equations. Freezing them makes a small error per step. More, smaller steps shrink the total error, but slowly: the average error falls like $\sqrt{\Delta t}$, so you need **4×** the steps to halve it. The dashed line on the right shows that rate.



Right panel: horizontal axis is the number of steps, doubling each tick. The orange dot is your current choice.

With 8 steps ($\Delta t = 1/8$), this path ends at 0.466 vs exact 0.412. Average error over 500 paths: 0.1533.

Quadrupling the steps from 2 cut the error by a factor of 1.75 (theory: about 2).

Euler-Maruyama converges to the true solution as the steps shrink, with error proportional to $\sqrt{\Delta t}$ (called "strong order $1/2$ "). It is the workhorse behind almost every simulation of an SDE, including every other chart on this page.

Going deeper: the precise statement

Convergence orders. If the coefficients a and b are Lipschitz, Euler–Maruyama has *strong* order $\frac{1}{2}$, meaning $\mathbb{E}|X_T - X_T^{(N)}| \leq C \Delta t^{1/2}$ (what the chart measures). It has *weak* order 1, meaning $|\mathbb{E}f(X_T) - \mathbb{E}f(X_T^{(N)})| \leq C \Delta t$ for smooth f . If you only need averages, as in option pricing, the method is more accurate than this chart suggests.

Doing better: the Milstein scheme. Add the next Itô–Taylor term, $\frac{1}{2} b(X_k) b'(X_k) ((\Delta W_k)^2 - \Delta t)$, and the strong order rises to 1. The correction contains $(\Delta W)^2 - \Delta t$: section 2.1 once more.

References: P. Kloeden & E. Platen, *Numerical Solution of Stochastic Differential Equations* (Springer, 1992). D. Higham, *SIAM Review* 43 (2001), 525–546.

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(13)
mu, sigma, paths, fine = 0.2, 0.8, 2_000, 1024
dW = rng.normal(0, np.sqrt(1 / fine), (paths, fine))
exact = np.exp((mu - sigma ** 2 / 2) + sigma * dW.sum(axis=1)) # exact GBM value at t = 1
for N in (8, 32, 128, 512):
    X = np.ones(paths)
    for incr in dW.reshape(paths, N, -1).sum(axis=2).T: # N coarse steps, same noise
        X += mu * X / N + sigma * X * incr
    print(f"N = {N:>3}: average error {np.mean(np.abs(X - exact)):.4f}") # about halves
    per 4x N
```

Output when we ran it:

```
N = 8: average error 0.1595
N = 32: average error 0.0848
N = 128: average error 0.0412
N = 512: average error 0.0203
```

5.2 A Zoo of Diffusions

Every SDE is a choice of drift a and volatility b . Three choices appear everywhere. Pick a model and drag its two settings. All three are driven by the same fixed random kicks, so what you see changing is the model, not the luck.

Predict first: In the OU model, make the pull strength θ four times larger. What will happen to the long-run spread?

4× narrower

2× narrower

Unchanged

Answer: "2× narrower". The long-run spread is $\sigma/\sqrt{2\theta}$, so 4× the pull gives $\sqrt{4} = 2\times$ narrower.

$$dX_t = a(X_t) dt + b(X_t) dW_t$$

dX_t The tiny change in X over a tiny time step dt (section 4.2).

$a(X_t) dt$ Drift: the predictable push, which depends on where X is now.

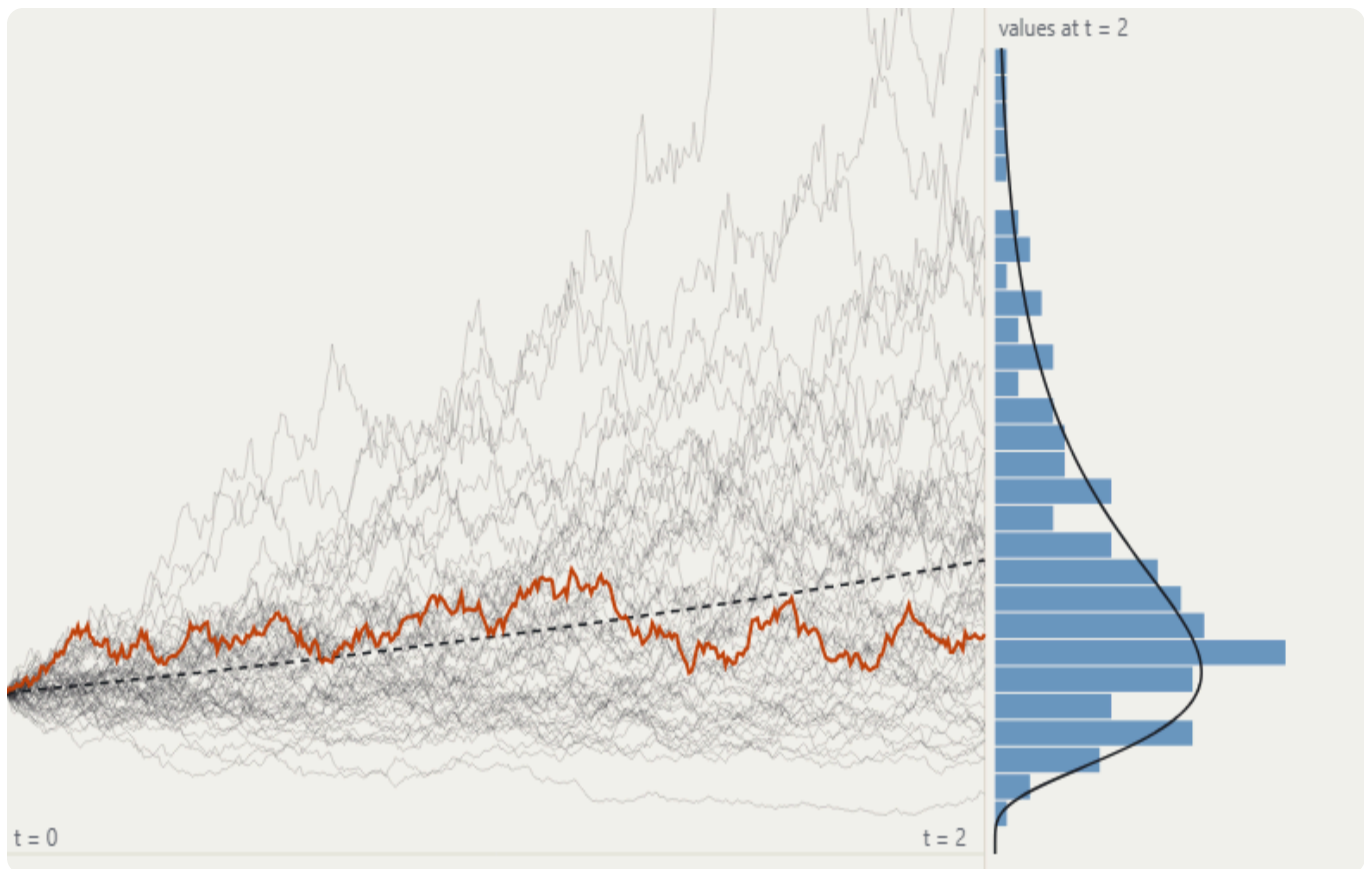
$b(X_t) dW_t$ Noise: a random kick whose size depends on where X is now.

GBM *Geometric Brownian motion*, $a = \mu x$, $b = \sigma x$. Grows and wobbles in percentage terms, never below 0. The standard model of a stock price.

OU *Ornstein–Uhlenbeck*, $a = \theta(0 - x)$, $b = \sigma$. A pull back toward 0 that gets stronger the further away X is, like a spring. Models velocities, interest rates and anything "mean-reverting".

CIR *Cox–Ingersoll–Ross*, $a = \kappa(1 - x)$, $b = \sigma\sqrt{x}$. Pulled toward 1, and the noise fades near 0. Models interest rates and volatility, which should not go negative.

In plain words: the drift decides where the paths want to go, and the volatility decides how wildly they get there. In GBM both scale with the value itself, so paths fan out upward. In OU the spring balances the noise, and the spread stops growing at a fixed width, $\sigma/\sqrt{2\theta}$. In CIR the noise shrinks near 0, so if the pull is strong enough compared with the noise ($2\kappa \geq \sigma^2$, the *Feller condition*), paths never reach 0 at all.



Grey: 60 of 200 paths (one in orange). Blue bars: where all 200 end at $t = 2$. Black curve: the exact end-value distribution, shown for GBM and OU where it has a simple formula.

Dashed: the average, $e^{\mu t}$. Paths spread out in proportion to their own size, so they fan out upward and never cross 0.

Three one-line equations give three very different kinds of randomness: exploding (GBM), settling (OU) and positive-with-a-floor (CIR). Choosing a and b to match what you are modelling is most of the art of applied stochastic calculus.

Going deeper: the precise statement

Exact solutions. GBM: $X_t = X_0 \exp\left((\mu - \frac{1}{2}\sigma^2)t + \sigma W_t\right)$ (from Itô's lemma applied to $\log X$). OU: $X_t = X_0 e^{-\theta t} + \sigma \int_0^t e^{-\theta(t-s)} dW_s$, which is normally distributed with the mean and variance drawn on the chart. CIR has no elementary formula, but X_t follows a scaled non-central chi-square distribution.

When does an SDE have a solution? If a and b are Lipschitz and grow at most linearly, there is exactly one solution (Itô's theorem). CIR's $\sqrt{x}$ is not Lipschitz at 0, and uniqueness there needs a finer result (Yamada–Watanabe). The Feller condition $2\kappa m \geq \sigma^2$ (here $m = 1$) is exactly what keeps 0 from ever being reached.

References: Øksendal, *Stochastic Differential Equations*, ch. 5. G. Uhlenbeck & L. Ornstein, *Phys. Rev.* 36 (1930), 823–841. J. Cox, J. Ingersoll & S. Ross, *Econometrica* 53 (1985), 385–407.

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(14)
paths, steps, T = 5_000, 400, 2.0
dt = T / steps
models = { # name: (drift a(x), volatility b(x), start, lowest allowed value)
    "GBM": (lambda x: 0.3 * x, lambda x: 0.4 * x, 1.0, -np.inf),
    "OU": (lambda x: 1.0 * (0 - x), lambda x: 0.6, 2.0, -np.inf),
    "CIR": (lambda x: 1.0 * (1 - x), lambda x: 0.5 * np.sqrt(x), 1.0, 0.0),
}
for name, (a, b, x0, floor) in models.items():
    X = np.full(paths, x0)
    for _ in range(steps):
        X = np.maximum(X + a(X) * dt + b(X) * rng.normal(0, np.sqrt(dt), paths), floor)
    print(f"{name}: mean at t = 2 is {X.mean():.3f}, spread {X.std():.3f}")
# theory: GBM mean  $e^{0.6} = 1.822$ ; OU mean  $2e^{-2} = 0.271$ , spread 0.420; CIR mean 1.000
```

Output when we ran it:

```
GBM: mean at t = 2 is 1.820, spread 1.147
OU: mean at t = 2 is 0.268, spread 0.415
CIR: mean at t = 2 is 1.000, spread 0.350
```

5.3 Paths ↔ Densities (Fokker–Planck)

There are two ways to describe an SDE. You can follow individual random *paths*, as above. Or you can describe the whole crowd at once with a *density*: a curve showing how likely each position is at each time. The density obeys an ordinary (non-random) equation, the *Fokker–Planck equation*. Here 3,000 particles follow the OU equation from near $x = 2$, while the computer separately solves the Fokker–Planck equation. Scrub the time or press Play.

Predict first: The crowd starts as a narrow spike near $x = 2$. What shape will it have at $t = 3$?

Still a narrow spike near 2

A wide bell curve centred near 0

Flat, spread out everywhere

Answer: "A wide bell curve centred near 0". The drift pulls the centre back toward 0 (like $2e^{(-t)}$), and the noise widens it until the spring balances it at spread $\sigma/\sqrt{2\theta} \approx 0.71$: a steady bell curve.

$$\frac{\partial p}{\partial t} = -\frac{\partial}{\partial x}(a(x)p) + \frac{1}{2} \frac{\partial^2}{\partial x^2}(b(x)^2 p)$$

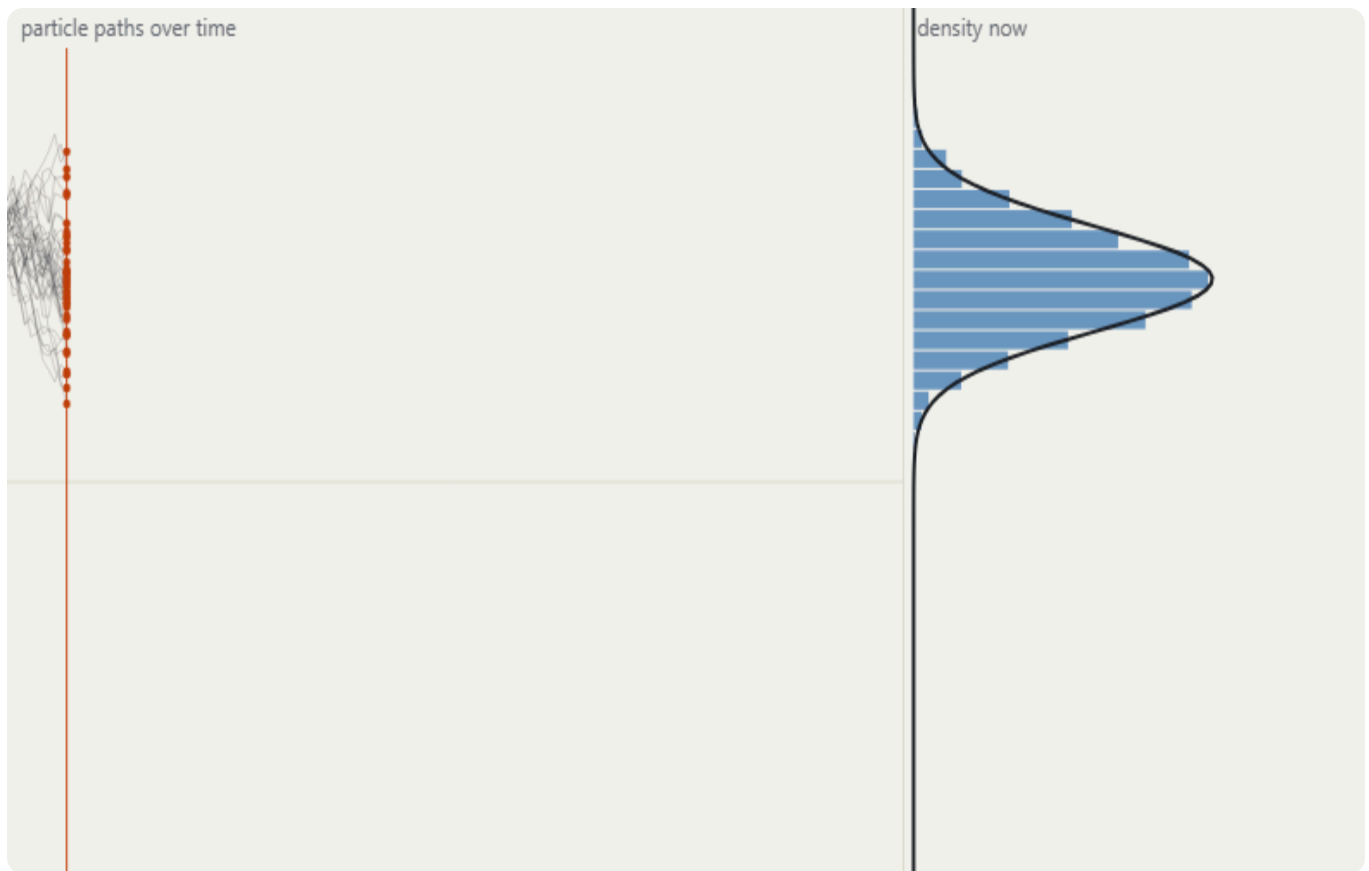
$p(t, x)$ The *density*: how crowded position x is at time t . The area under the curve between two positions is the fraction of particles between them.

$\frac{\partial p}{\partial t}$ How fast the density at one fixed position is changing over time. The curly ∂ ("partial") means only one variable changes while the others are held fixed.

$\frac{\partial}{\partial x}, \frac{\partial^2}{\partial x^2}$ The slope, and the curvature, of a curve drawn across positions at a fixed time.

$a(x), b(x)$ The drift and volatility from section 5.2. Here (OU): $a(x) = -x, b(x) = 1$.

In plain words: the first term *moves* the crowd: drift carries density along like a current, here back toward 0. The second term *spreads* it: noise smooths out peaks the way heat spreads through a metal bar. Density flows out of crowded, peaked regions (negative curvature) into sparse ones. Together they give one equation that predicts the shape of a crowd of 3,000 random paths without simulating any of them. The black curve (the equation) and the blue bars (the particles) should track each other at every moment.



Left: 40 of the 3,000 particles up to the current time (orange line). Right: histogram of all 3,000 particles now (blue) and the Fokker–Planck solution (black).

$t = 0.20$. Average position: particles 1.640, equation 1.637, exact 1.637. Spread (standard deviation): particles 0.425, equation 0.424, exact 0.424.

Random paths and a smooth equation tell the same story from two sides. This bridge between probability and differential equations runs through all of applied mathematics. The last section of this site, Feynman–Kac, crosses it in the opposite direction.

Going deeper: the precise statement

Derivation from Itô's lemma. For any smooth test function φ , Itô's lemma and taking averages give $\frac{d}{dt} \mathbb{E} \varphi(X_t) = \mathbb{E} \left[a(X_t) \varphi'(X_t) + \frac{1}{2} b(X_t)^2 \varphi''(X_t) \right]$. Write both sides as integrals against the density $p(t, x)$ and integrate by parts. Since φ was arbitrary, the Fokker–Planck equation follows. It is also called the *forward Kolmogorov equation*.

Steady state. Setting $\partial_t p = 0$ for OU gives the normal density $N(0, \sigma^2/2\theta)$, the bell curve the crowd settles into. The *backward* Kolmogorov equation, its mirror image, is the equation behind Feynman–Kac (section 6.3).

References: H. Risken, *The Fokker–Planck Equation* (2nd ed., Springer, 1989). Øksendal, *Stochastic Differential Equations*, ch. 8 (the generator and Kolmogorov's backward equation).

Show the code (Python + NumPy)

```
import numpy as np
rng = np.random.default_rng(15)
theta, sigma, x0, T = 1.0, 1.0, 2.0, 1.0
X = x0 + 0.15 * rng.normal(size=20_000) # particles
for _ in range(200): # 200 Euler-Maruyama steps of 0.005
    X += -theta * X * 0.005 + sigma * np.sqrt(0.005) * rng.normal(size=X.size)
x = np.linspace(-4, 5, 181) # Fokker-Planck on a grid
dx, dt = x[1] - x[0], 0.001
p = np.exp(-0.5 * ((x - x0) / 0.15) ** 2) / (0.15 * np.sqrt(2 * np.pi))
for _ in range(1000): # dp/dt = d/dx(theta x p) + 1/2
    sigma^2 d2p/dx2
    flux = theta * x * p
    p[1:-1] += dt * ((flux[2:] - flux[:-2]) / (2 * dx) + 0.5 * sigma ** 2 * (p[2:] - 2 *
p[1:-1] + p[:-2]) / dx ** 2)
print(f"t = 1: particle mean {X.mean():.3f}, PDE mean {np.sum(x * p) * dx:.3f}, exact {x0 *
np.exp(-theta * T):.3f}")
```

Output when we ran it:

```
t = 1: particle mean 0.738, PDE mean 0.735, exact 0.736
```

Chapter recap

Quick questions on this chapter's big ideas. Each answer is explained as soon as you choose.

1. Euler–Maruyama's average error shrinks like $\sqrt{\Delta t}$. Going from 100 to 400 steps, the error...

Halves

Falls to a quarter

Stays the same

Answer: "Halves". Four times as many steps makes Δt four times smaller, and $\sqrt{4} = 2$.

2. In the Ornstein–Uhlenbeck model, what happens to the long-run spread if the noise σ doubles?

It doubles

It quadruples

It halves

Answer: "It doubles". The long-run spread is $\sigma/\sqrt{2\theta}$, which is proportional to σ .

3. What does the Fokker–Planck equation describe?

One particular random path

How the density of many paths changes over time

Only the drift, without the noise

Answer: "How the density of many paths changes over time". It is a deterministic equation for the density $p(t, x)$ of the crowd of paths, with a drift term and a spreading term.